Bima Kwik System Design Document (Detailed)

1. Introduction

This document provides a detailed system design for Bima Kwik, a digital insurance platform. It expands on the previous document, providing a comprehensive overview of the architecture, UI/UX, and a breakdown of each module.

2. System Architecture

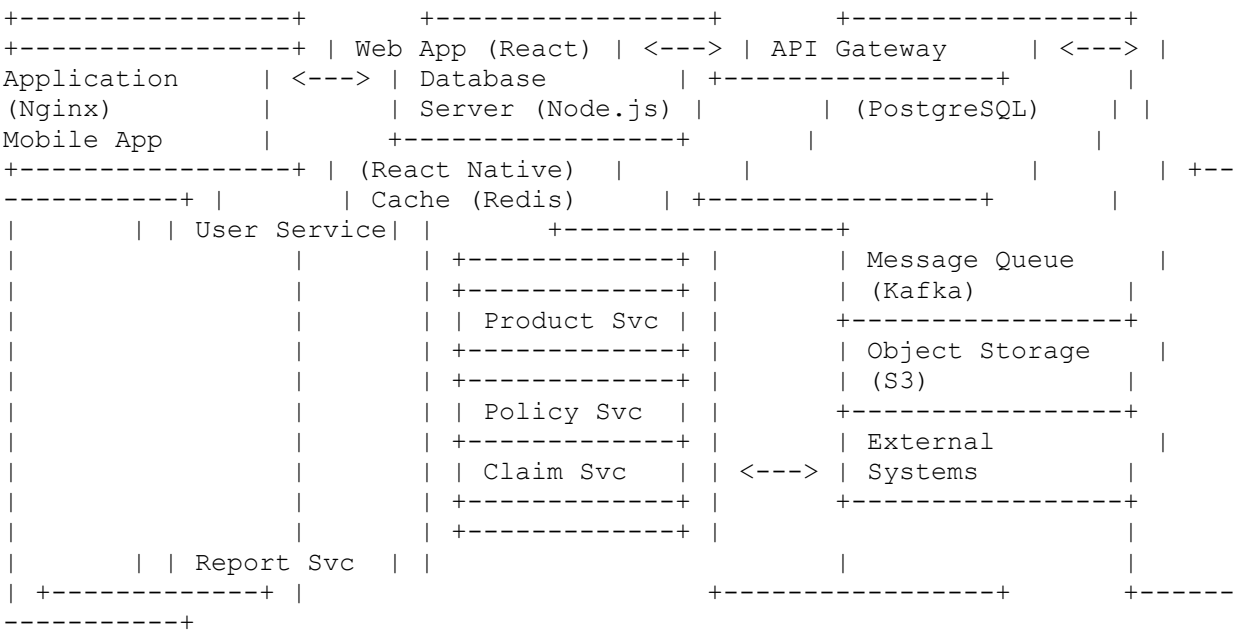
2.1 Overview

Bima Kwik employs a multi-tiered architecture to ensure scalability, maintainability, and security. The system is divided into the following layers:

- **Presentation Layer:** Handles the user interface for web and mobile applications.
- **Application Layer:** Contains the business logic and API endpoints.
- **Data Layer:** Manages data storage and retrieval.
- **Integration Layer:** Facilitates communication with external systems.

2.2 Architecture Diagram

[A diagram illustrating the multi-tiered architecture with components. A more detailed version of the previous diagram, possibly including message queues, caching, and specific services. For example:]



2.3 Component Description

- **2.3.1 Presentation Layer**
  - **Web Application:**
    - Technology: React
    - Description: Provides a web-based interface for users to interact with Bima Kwik.
    - Features:
      - User authentication and authorization
      - Product catalog and search
      - Policy management
      - Claims submission and tracking
      - Reporting and dashboards
  - **Mobile Applications:**
    - Technology: React Native{***flutter***}
    - Description: Offers mobile access to Bima Kwik functionalities for customers and agents.
    - Features:
      - Customer App:
        - Policy viewing and management
        - Claims submission
        - Notifications
        - Payment
      - Agent App:
        - Customer management
        - Product information
        - Quotation generation
        - Policy issuance
        - Commission tracking
- **2.3.2 Application Layer**
  - **API Gateway:**
    - Technology: Nginx
    - Description: Serves as the entry point for all client requests, routing them to the appropriate backend services.
    - Features:
      - Request routing
      - Authentication and authorization
      - Rate limiting
      - Load balancing
      - API composition
      - CORS handling
  - **Application Services:**
    - Description: A suite of microservices responsible for specific business functions.
    - Technology: Node.js with Express.js{**laravel**}
    - Services:

        - **User Service:** Manages user accounts, authentication, and authorization.
        - **Product Service:** Handles insurance product catalog, pricing, and recommendations.
        - **Policy Service:** Manages policy creation, issuance, renewal, and endorsements.
        - **Claim Service:** Processes and manages insurance claims.
        - **Report Service:** Generates various reports (financial, operational).
- **2.3.3 Data Layer**
    - **Database:**
        - Technology: PostgreSQL
        - Description: Stores persistent data for the application.
        - Features:
            - Relational database schema
            - Data integrity and consistency
            - Scalability and performance
            - Backup and recovery
    - **Cache:**
        - Technology: Redis
        - Description: Caches frequently accessed data to improve performance.
        - Features
            - In-memory data storage
            - Key-value store
            - Session management
    - **Message Queue:**
        - Technology: Apache Kafka
        - Description: Enables asynchronous communication between services.
        - Features:
            - Reliable messaging
            - Scalability
            - Fault tolerance
- **2.3.4 Integration Layer**
    - **External Systems:**
        - Description: Integrates with external services.
        - Examples:
            - Payment gateways (e.g., PayPal, Stripe)
            - Insurance provider systems (APIs)
            - Government agencies (for ID verification)
            - SMS/Email gateways
            - AI services

3. User Interface (UI) and User Experience (UX) Design

3.1 UI Design Principles

- **Clean and Modern:** A visually appealing design with a focus on clarity and simplicity.

- **Responsive:** Adapts seamlessly to different screen sizes (desktop, tablet, mobile).
- **Consistent:** Consistent use of colors, fonts, and UI elements across all platforms.
- **Accessible:** Adheres to accessibility guidelines (WCAG) to ensure usability for all users.
- **Branding:** Reflects the Bima Kwik brand identity.

3.2 UX Design Principles

- **User-Centered:** Design focused on user needs and goals.
- **Intuitive:** Easy to navigate and understand.
- **Efficient:** Streamlines insurance processes and minimizes user effort.
- **Personalized:** Provides customized experiences based on user roles and preferences.
- **Secure:** Protects user data and ensures secure transactions.

3.3 User Flows

[Detailed diagrams illustrating key user flows, including:

- Customer registration and policy purchase
- Agent quotation and policy issuance
- Claims submission and tracking
- Admin dashboard and reporting
- Service provider registration and management

  Include steps, decisions, and interactions.

  ]

3.4 Wireframes and Mockups

[Detailed wireframes and mockups of key screens for the web and mobile applications, showing layout, content, and functionality.]

3.5 Style Guide

[Comprehensive style guide defining the visual style of the application.]

4. Module Design

4.1 User Management Module

- **Description:** Manages user accounts, authentication, and authorization.
- **Components:**
  - Registration: User registration process with validation.
  - Login: User authentication using username/password, social login.
  - Profile Management: User profile creation, viewing, and editing.

    - Role Management: Assigning roles (customer, agent, broker, admin) and permissions.
    - Password Management: Password reset, change password.
    - Session Management: Handling user sessions and security.
- **API Endpoints:**
    - `POST /api/users/register`: Register a new user.
    - `POST /api/users/login`: Authenticate a user and return a token.
    - `GET /api/users/me`: Get the current user's profile.
    - `PUT /api/users/me`: Update the current user's profile.
    - `POST /api/users/password/reset`: Request a password reset.
    - `POST /api/users/password/change`: Change user password.
    - `GET /api/users/{id}`: Get a specific user.
    - `GET /api/users`: Get all users (admin).
- **Data Models:**
    - User: (id, username, email, password, role, profile, created_at, updated_at)
    - Role: (id, name, description)
    - Permission: (id, name, description)

4.2 Product Management Module

- **Description:** Manages insurance products, including types, details, and pricing.
- **Components:**
    - Product Catalog: Listing available insurance products.
    - Product Details: Viewing detailed information about a product.
    - Product Search: Searching for products by criteria.
    - Recommendations: Providing product recommendations to users.
    - Pricing: Calculating premiums based on user input and product rules.
- **API Endpoints:**
    - `GET /api/products`: Get all insurance products.
    - `GET /api/products/{id}`: Get a specific product.
    - `GET /api/products/types`: Get all insurance types.
    - `POST /api/products/recommendations`: Get product recommendations
- **Data Models:**
    - Product: (id, name, description, type_id, premium, benefits, exclusions, coverage_area, age_range, details)
    - ProductType: (id, name, description)

4.3 Policy Management Module

- **Description**: Manages insurance policies, including creation, issuance, renewal, and endorsements.
- **Components**:
    - Policy Creation: Creating new policies based on product selection and customer data.
    - Policy Issuance: Generating policy documents and issuing them to customers.
    - Policy Renewal: Managing policy renewal process.

    - Policy Endorsement: Handling changes to existing policies.
    - Policy Viewing: Allowing users to view their policies.
- **API Endpoints:**
    - `POST /api/policies`: Create a new policy.
    - `GET /api/policies/{id}`: Get a specific policy.
    - `GET /api/policies/user/{userId}`: Get policies for a user.
    - `PUT /api/policies/{id}/renew`: Renew a policy.
    - `PUT /api/policies/{id}/endorse`: Endorse a policy
- **Data Models**:
    - Policy: (id, user_id, product_id, start_date, end_date, premium, status, details)

4.4 Transaction Management Module

- **Description:** Handles financial transactions, including premium payments and commission calculations.
- **Components:**
    - Payment Processing: Integrating with payment gateways to process payments.
    - Payment Confirmation: Generating payment receipts and notifications.
    - Commission Calculation: Calculating agent/broker commissions.
    - Transaction History: Viewing transaction history.
- **API Endpoints:**
    - `POST /api/transactions/payment`: Process a payment.
    - `GET /api/transactions/{id}`: Get a specific transaction.
    - `GET /api/transactions/user/{userId}`: Get transactions for a user
    - `GET /api/transactions`: Get all transactions (admin).
- **Data Models:**
    - Transaction: (id, user_id, type, amount, status, details, created_at)

4.5 Claims Management Module

- **Description:** Manages the process of submitting, tracking, and processing insurance claims.
- **Components:**
    - Claim Submission: Allowing users to submit claims with supporting documents.
    - Claim Tracking: Tracking the status of submitted claims.
    - Claim Processing: Processing claims by insurance providers.
    - Claim Settlement: Managing claim payouts.
- **API Endpoints:**
    - `POST /api/claims`: Submit a new claim.
    - `GET /api/claims/{id}`: Get a specific claim.
    - `GET /api/claims/user/{userId}`: Get claims for a user.
    - `PUT /api/claims/{id}/status`: Update claim status.
- **Data Models:**
    - Claim: (id, policy_id, claimant_id, date, status, amount, details)

4.6 Report Management Module

- **Description:** Generates reports on various aspects of the business, such as transactions, claims, and user activity.
- **Components:**
  - Report Generation: Creating reports based on specified parameters.
  - Report Viewing: Displaying reports in various formats.
  - Report Scheduling: Scheduling reports to be generated automatically.
- **API Endpoints:**
  - `GET /api/reports/{type}`: Get a specific report.
- **Data Models:**
  - Report: (id, type, date, description, data) // 'data' can be JSON

4.7 AI Integration Module

- **Description:** Integrates AI services for risk assessment, fraud detection, and other intelligent features.
- **Components:**
  - Risk Assessment: Using AI to evaluate risk associated with policies.
  - Fraud Detection: Identifying potentially fraudulent claims.
  - Personalized Recommendations
- **API Endpoints:**
  - `POST /api/ai/risk-assessment`: Assess risk for a policy.
  - `POST /api/ai/fraud-detection`: Detect fraud in a claim.

4.8 Document Management Module

- **Description**: Handles uploading, storing, and retrieving documents related to policies, claims, and customers.
- **Components**:
  - Document Upload
  - Document Storage
  - Document Retrieval
  - Document Indexing
- **API Endpoints**:
  - `POST /api/documents`: Upload a document.
  - `GET /api/documents/{id}`: Retrieve a document.

4.9 Service Provider Management Module

- **Description**: Manages service providers (hospitals, garages, etc.)
- **Components**:
  - Provider Registration
  - Provider Search
  - Provider Details
  - Provider Rating

- **API Endpoints**:
    - `POST /api/service-providers`: Register a service provider.
    - `GET /api/service-providers`: Get all service providers.

5. Technology Stack

[Detailed technology stack, including specific versions and configurations.]

6. Security Considerations

[Detailed security measures, including authentication, authorization, data encryption, and vulnerability testing.]

7. Scalability and Performance

[Detailed strategies for ensuring scalability and performance, including load balancing, caching, and database optimization.]

8. Integration

[Detailed information on API design, authentication, data formats, and error handling for integrating with external systems.]

Bima Kwik System Design Document - Expanded Sections

5-8

Here's a more detailed explanation of sections 5, 6, 7, and 8 from the Bima Kwik System Design Document:

5. Technology Stack

This section details the specific technologies, frameworks, and tools used to build Bima Kwik.

- **Front-End:**
    - **Web:**
        - Language: TypeScript
        - Framework: React (with Next.js for server-side rendering)
        - State Management: Redux Toolkit
        - UI Library: Shadcn/UI
        - Build Tool: Vite
    - **Mobile:**
        - Framework: React Native
        - State Management: Redux Toolkit
        - UI Library: React Native Paper

        - Build Tool: Expo
- **Back-End:**
    - Language: Node.js (TypeScript)
    - Framework: Express.js
    - Database ORM: Prisma
    - Real-time Communication: WebSockets (Socket.IO)
- **Database:**
    - Technology: PostgreSQL
    - Version: 14 or later
    - Extensions: PostGIS (for geospatial data, if needed)
- **Cache:**
    - Technology: Redis
    - Deployment: ElastiCache (AWS) or similar
- **Message Queue:**
    - Technology: Apache Kafka
    - Deployment: Confluent Cloud or self-hosted
- **API Gateway:**
    - Technology: Nginx
    - Configuration: Reverse proxy, load balancing, security headers
- **Cloud Platform:**
    - Provider: AWS (Amazon Web Services)
    - Services:
        - EC2 (for application servers)
        - RDS (for PostgreSQL)
        - ElastiCache (for Redis)
        - Kafka (Confluent Cloud or Managed Service)
        - S3 (for object storage)
        - CloudFront (for CDN)
        - Route 53 (for DNS)
        - IAM (for access management)
        - CloudWatch (for monitoring and logging)
- **AI/ML (If Applicable):**
    - Language: Python
    - Frameworks: TensorFlow, PyTorch
    - Platform: SageMaker (AWS)
    - Libraries: scikit-learn, Pandas, NumPy
- **Testing:**
    - Unit Testing: Jest
    - Integration Testing: Jest, Supertest
    - End-to-End Testing: Cypress
    - Performance Testing: LoadView, JMeter
    - Mobile Testing: React Native Testing Library
- **DevOps:**
    - Infrastructure as Code: Terraform
    - Containerization: Docker
    - Orchestration: Kubernetes (EKS on AWS)

    - CI/CD: GitHub Actions, Jenkins
    - Monitoring and Logging: CloudWatch, Prometheus, Grafana
    - Alerting: PagerDuty, SNS (AWS)

6. Security Considerations

This section outlines the security measures implemented to protect the Bima Kwik system and user data.

- **Authentication:**
    - Method: JSON Web Tokens (JWT) for API authentication
    - Strategy:
        - Users authenticate with username/password or social providers (OAuth 2.0).
        - Upon successful authentication, the server issues a JWT.
        - The client includes the JWT in the `Authorization` header for subsequent requests.
        - The server verifies the JWT signature and expiration for each request.
    - Token Storage: Securely stored on the client-side (e.g., in `localStorage` or secure HTTP-only cookies).
    - Multi-factor Authentication (MFA): Optional, but recommended for sensitive operations.
- **Authorization:**
    - Method: Role-Based Access Control (RBAC)
    - Strategy:
        - Users are assigned roles (e.g., "customer," "agent," "admin").
        - Roles are associated with a set of permissions (e.g., "view-policies," "create-claims").
        - The server checks the user's role and permissions before allowing access to resources or performing actions.
    - Fine-grained authorization: Implement attribute-based access control (ABAC) for more complex scenarios.
- **Data Encryption:**
    - In Transit:
        - Protocol: HTTPS (TLS 1.2 or later) for all communication between clients and servers.
        - Certificates: Valid SSL/TLS certificates from a trusted Certificate Authority.
        - HSTS: HTTP Strict Transport Security to enforce HTTPS.
    - At Rest:
        - Database: Transparent Data Encryption (TDE) for sensitive data (e.g., personally identifiable information - PII).
        - Object Storage: Server-Side Encryption (SSE) for files stored in S3.
        - Configuration: Securely manage encryption keys using a key management service (e.g., AWS KMS).
- **Vulnerability Testing:**

    - Frequency:
        - Regular security scans (e.g., using OWASP ZAP) as part of the CI/CD pipeline.
        - Annual penetration testing by a third-party security firm.
        - Ongoing monitoring for security vulnerabilities.
    - Types:
        - Static Application Security Testing (SAST)
        - Dynamic Application Security Testing (DAST)
        - Penetration Testing
        - Dependency scanning (e.g., using OWASP Dependency-Check)
- **Input Validation:**
    - Server-Side: Strict validation of all input data on the server-side to prevent injection attacks (e.g., SQL injection, command injection).
    - Client-Side: Client-side validation for a better user experience, but server-side validation is always the source of truth.
    - Techniques:
        - Use of parameterized queries or prepared statements.
        - Input sanitization and escaping.
        - Schema validation for API requests.
- **Protection Against Common Attacks:**
    - Cross-Site Request Forgery (CSRF): Use anti-CSRF tokens (e.g., Synchronizer Token Pattern).
    - Cross-Site Scripting (XSS): Properly escape user-generated content.
    - SQL Injection: Use parameterized queries.
    - Denial of Service (DoS): Implement rate limiting and request throttling.
    - Man-in-the-Middle (MitM): Enforce HTTPS and use HSTS.
- **Logging and Monitoring**
    - Comprehensive logging of security-related events (authentication failures, authorization errors, etc.)
    - Real-time monitoring for suspicious activity.
    - Centralized log management (e.g., using Elasticsearch, Logstash, and Kibana - ELK stack).
- **Data Privacy**
    - Compliance with relevant data privacy regulations (e.g., GDPR).
    - Implement data minimization principles.
    - Provide users with control over their data.
    - Have a clear data retention policy.

7. Scalability and Performance

This section details the strategies for ensuring Bima Kwik can handle increasing user loads and maintain optimal performance.

- **Load Balancing:**
    - API Gateway: Nginx will distribute incoming traffic across multiple application server instances.

    - Application Servers: Horizontal scaling by adding more EC2 instances as needed.
    - Database: Read replicas for the database to handle read-heavy traffic.
- **Caching:**
    - CDN: Amazon CloudFront to cache static assets (images, CSS, JavaScript) and reduce latency for users.
    - Server-Side Caching: Redis to cache frequently accessed data (e.g., product catalogs, user profiles).
    - Database Caching: Query caching at the application level and database level.
    - HTTP Caching: Use appropriate HTTP caching headers (e.g., `Cache-Control`, `ETag`).
- **Database Optimization:**
    - Indexing: Properly index database tables to improve query performance.
    - Query Optimization: Write efficient SQL queries and use query optimization tools.
    - Connection Pooling: Use connection pooling to reduce the overhead of establishing database connections.
    - Database Sharding: Consider database sharding for very large datasets.
- **Asynchronous Processing:**
    - Message Queue: Apache Kafka to handle long-running or non-time-critical tasks asynchronously (e.g., sending email notifications, processing claims).
    - Benefits: Improves responsiveness, reduces load on the main application servers.
- **Horizontal Scaling:**
    - Application Servers: Scale horizontally by adding more EC2 instances behind the load balancer.
    - Database: Use read replicas and consider sharding for write-heavy workloads.
    - Microservices: The microservices architecture allows for independent scaling of individual services.
- **Performance Monitoring:**
    - Metrics: Collect key performance metrics (e.g., response time, throughput, error rate, CPU usage, memory usage).
    - Tools: Amazon CloudWatch, Prometheus, Grafana.
    - Alerting: Set up alerts to notify the operations team of performance issues.
    - Regular Performance Testing: Conduct load tests to identify bottlenecks and ensure the system can handle expected traffic.

8. Integration

This section provides details on how Bima Kwik integrates with external systems.

- **API Design:**
    - Style: RESTful API
    - Principles:
        - Use of standard HTTP methods (GET, POST, PUT, DELETE).
        - Resource-based URLs (e.g., `/api/users`, `/api/policies/{id}`).
        - Use of nouns, not verbs, in URLs
        - Statelessness

        - HATEOAS (Hypermedia As The Engine Of Application State) - optional, but good practice
- **Authentication:**
    - Methods:
        - JWT (JSON Web Tokens) for internal service-to-service communication.
        - OAuth 2.0 for third-party applications (if needed).
        - API Keys for partner integrations (e.g., with insurance providers).
    - Security: Securely store API keys and credentials.
- **Data Formats:**
    - Request/Response: JSON (JavaScript Object Notation)
    - File Transfer: Multipart/form-data (for file uploads)
- **Error Handling:**
    - HTTP Status Codes: Use standard HTTP status codes to indicate the result of an API request (e.g., 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 500 Internal Server Error).
    - Error Responses: Provide detailed error messages in the response body, including an error code and a human-readable message.
    - Logging: Log all API requests and errors for debugging and monitoring.
- **Rate Limiting:**
    - Purpose: Protect the API from abuse and ensure fair usage.
    - Implementation: Limit the number of requests a client can make within a given time period.
    - Headers: Include rate limit headers in the API responses (e.g., `X-RateLimit-Limit`, `X-RateLimit-Remaining`, `X-RateLimit-Reset`).
- **Documentation:**
    - Format: OpenAPI Specification (Swagger)
    - Content:
        - Detailed description of each endpoint.
        - Request parameters and body.
        - Response formats and examples.
        - Authentication and authorization requirements.
        - Error codes and messages.
    - Accessibility: Make the API documentation easily accessible to developers.
- **Versioning:**
    - Strategy: Use URI versioning (e.g., `/api/v1/users`, `/api/v2/users`).
    - Rationale: Allows for backward-compatible changes and easier management of API updates.
- **Idempotency:**
    - Design APIs to be idempotent where appropriate (e.g., PUT, DELETE). This means that making the same request multiple times has the same effect as making it once.